



Redux POS Dashboard Lab: Inventory, Cart & Sales History

MERN Stack Bootcamp – Professional Redux Practice

Instructor: Kashif Raza | [LinkedIn](#)

Duration: ~2.5–3 hours

Difficulty: Beginner → Intermediate



Lab Overview

You will build a **Retail POS Dashboard** that manages product inventory, handles real-time cart operations, and logs completed sales. This lab replaces prop-drilling and scattered `useState` with **Redux Toolkit (RTK)**, the industry standard for predictable, debuggable state.



Why a POS Dashboard?

Retail, e-commerce, and SaaS apps all rely on the same Redux patterns: shared cart state, inventory updates across components, async order processing, and dashboard metrics. Master this, and you've mastered real-world Redux.



Quick Setup

```
npm create vite@latest redux-pos-lab -- --template react
cd redux-pos-lab
npm install @reduxjs/toolkit react-redux lucide-react
npm run dev
```

Professional Folder Structure

```
src/
├─ main.jsx           # Entry point + Provider
├─ App.jsx            # Dashboard layout
├─ store/
│   ├─ index.js       # configureStore
│   └─ slices/
│       ├─ inventorySlice.js # Products, stock levels
│       ├─ cartSlice.js     # Current transaction
│       └─ salesSlice.js    # Completed orders, metrics
├─ components/
│   ├─ ProductGrid.jsx
│   ├─ CartPanel.jsx
│   ├─ SalesLog.jsx
│   └─ DashboardStats.jsx
└─ App.css / index.css # Base styles (Tailwind or custom)
```

◆ Phase 1: Store Setup & Core Slices (40 mins)

🎯 Goal

Create a Redux store, build inventory & cart slices, and verify state flow in DevTools.

📄 Step 1.1: Inventory Slice (src/store/slices/inventorySlice.js)

```
import { createSlice } from '@reduxjs/toolkit';

const initialState = [
  { id: 1, name: 'Wireless Mouse', price: 29.99, stock: 15, category: 'tech' },
  { id: 2, name: 'USB-C Cable', price: 9.99, stock: 42, category: 'tech' },
  { id: 3, name: 'Notebook A5', price: 4.50, stock: 8, category: 'office' },
  { id: 4, name: 'Desk Lamp', price: 34.99, stock: 3, category: 'office' },
];

export const inventorySlice = createSlice({
  name: 'inventory',
  initialState,
  reducers: {
    updateStock: (state, action) => {
      const { id, quantity } = action.payload;
      const item = state.find(p => p.id === id);
      if (item) item.stock = Math.max(0, item.stock - quantity);
    },
    restock: (state, action) => {
      const { id, quantity } = action.payload;
      const item = state.find(p => p.id === id);
      if (item) item.stock += quantity;
    },
  },
});

export const { updateStock, restock } = inventorySlice.actions;
export default inventorySlice.reducer;
```

📄 Step 1.2: Cart Slice (src/store/slices/cartSlice.js)

```
import { createSlice } from '@reduxjs/toolkit';

const initialState = {
  items: [],
  total: 0,
};

export const cartSlice = createSlice({
  name: 'cart',
  initialState,
  reducers: {
    addItem: (state, action) => {
      const product = action.payload;
      const existing = state.items.find(i => i.id === product.id);
      if (existing) {
        existing.quantity += 1;
      } else {
        state.items.push({ ...product, quantity: 1 });
      }
    },
    removeItem: (state, action) => {
```

```

    state.items = state.items.filter(i => i.id !== action.payload);
  },
  updateQuantity: (state, action) => {
    const { id, quantity } = action.payload;
    const item = state.items.find(i => i.id === id);
    if (item) item.quantity = Math.max(1, quantity);
  },
  clearCart: (state) => {
    state.items = [];
    state.total = 0;
  },
  calculateTotal: (state) => {
    state.total = state.items.reduce((sum, i) => sum + i.price * i.quantity, 0);
  },
},
});

export const { addItem, removeItem, updateQuantity, clearCart, calculateTotal } =
cartSlice.actions;
export default cartSlice.reducer;

```

💡 **RTK Secret:** You're writing "mutating" code, but Immer (built into RTK) safely clones state behind the scenes. No more ``...state`` or ``map()`` just to update one field.

Step 1.3: Configure Store (`src/store/index.js`)

```

import { configureStore } from '@reduxjs/toolkit';
import inventoryReducer from './slices/inventorySlice';
import cartReducer from './slices/cartSlice';
// salesReducer added in Phase 3

export const store = configureStore({
  reducer: {
    inventory: inventoryReducer,
    cart: cartReducer,
  },
});

```

✅ **Checkpoint:** Open Redux DevTools → dispatch `addItem` manually → watch `cart.items` update instantly. No props passed. No state lifting.

◆ Phase 2: Connect UI to Redux (45 mins)

🎯 Goal

Use `useSelector` & `useDispatch` to build components that read/write shared state cleanly.

📄 Step 2.1: Product Grid (`src/components/ProductGrid.jsx`)

```
import { useDispatch, useSelector } from 'react-redux';
import { addItem } from '../store/slices/cartSlice';
import { updateStock } from '../store/slices/inventorySlice';

export default function ProductGrid() {
  const dispatch = useDispatch();
  const products = useSelector(state => state.inventory);

  const handleAdd = (product) => {
    if (product.stock <= 0) return alert('Out of stock!');
    dispatch(addItem(product));
    dispatch(updateStock({ id: product.id, quantity: 1 }));
  };

  return (
    <div className="bg-white p-4 rounded shadow">
      <h2 className="font-bold mb-3">Inventory</h2>
      <div className="grid grid-cols-2 gap-3">
        {products.map(p => (
          <div key={p.id} className={`p-3 border rounded ${p.stock <= 5 ? 'border-red-300 bg-red-50' : 'border-gray-200'}>
            <h3 className="font-medium">{p.name}</h3>
            <p className="text-sm text-gray-500">${p.price} • Stock: {p.stock}</p>
            <button
              onClick={() => handleAdd(p)}
              disabled={p.stock <= 0}
              className="mt-2 w-full bg-indigo-600 text-white py-1 rounded text-sm disabled:opacity-50"
            >
              {p.stock <= 0 ? 'Out of Stock' : 'Add to Cart'}
            </button>
          </div>
        ))}
      </div>
    </div>
  );
}
```

📄 Step 2.2: Cart Panel (`src/components/CartPanel.jsx`)

```
import { useDispatch, useSelector } from 'react-redux';
import { removeItem, updateQuantity, clearCart, calculateTotal } from '../store/slices/cartSlice';

export default function CartPanel() {
  const dispatch = useDispatch();
  const { items, total } = useSelector(state => state.cart);

  return (
    <div className="bg-white p-4 rounded shadow flex flex-col h-full">
      <h2 className="font-bold mb-3">Current Sale</h2>
      <div className="flex-1 overflow-y-auto max-h-64 space-y-2 mb-3">
        {items.length === 0 ? (

```

```

        <p className="text-gray-500 text-center py-4">Cart is empty</p>
      ) : items.map(item => (
        <div key={item.id} className="flex justify-between items-center border-b pb-2">
          <div>
            <p className="font-medium text-sm">{item.name}</p>
            <p className="text-xs text-gray-500">${item.price} x {item.quantity}</p>
          </div>
          <div className="flex gap-1">
            <button onClick={() => dispatch(updateQuantity({ id: item.id, quantity:
item.quantity - 1 })))} className="px-2 bg-gray-200 rounded">-</button>
            <button onClick={() => dispatch(updateQuantity({ id: item.id, quantity:
item.quantity + 1 })))} className="px-2 bg-gray-200 rounded">+</button>
            <button onClick={() => dispatch(removeItem(item.id))} className="ml-1 text-red-
500">x</button>
          </div>
        </div>
      )})
    </div>
    <div className="border-t pt-3">
      <div className="flex justify-between font-bold text-lg mb-3">
        <span>Total</span><span>${total.toFixed(2)}</span>
      </div>
      <button onClick={() => dispatch(clearCart())} className="w-full text-red-600 text-sm
hover:underline">Clear Cart</button>
    </div>
  </div>
);
}

```

Step 2.3: Wire it in src/App.jsx

```

import { useDispatch, useSelector } from 'react-redux';
import ProductGrid from './components/ProductGrid';
import CartPanel from './components/CartPanel';
import { calculateTotal } from './store/slices/cartSlice';

export default function App() {
  const dispatch = useDispatch();
  const items = useSelector(state => state.cart.items);

  // Auto-calculate total whenever cart changes
  useEffect(() => {
    if (items.length > 0) dispatch(calculateTotal());
  }, [items, dispatch]);

  return (
    <div className="min-h-screen bg-gray-50 p-6 max-w-6xl mx-auto">
      <h1 className="text-2xl font-bold mb-6">🛒 POS Dashboard</h1>
      <div className="grid md:grid-cols-3 gap-6">
        <div className="md:col-span-2">ProductGrid </div>
        <div className="md:col-span-1">CartPanel </div>
      </div>
    </div>
  );
}

```

✅ **Checkpoint:** Add products → stock decreases → cart updates → total recalculates. Open DevTools → see clean action log. Zero prop drilling.

◆ Phase 3: Async Sales & History (50 mins)

🎯 Goal

Process sales asynchronously, log them to history, and update dashboard metrics.

📄 Step 3.1: Sales Slice with Thunk (src/store/slices/salesSlice.js)

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

export const completeSale = createAsyncThunk('sales/complete', async (_, { getState,
rejectWithValue }) => {
  const cart = getState().cart;
  if (cart.items.length === 0) return rejectWithValue('Cart is empty');

  // Simulate backend processing
  await new Promise(res => setTimeout(res, 800));

  return {
    id: Date.now(),
    items: cart.items,
    total: cart.total,
    timestamp: new Date().toISOString(),
    status: 'completed',
  };
});

const salesSlice = createSlice({
  name: 'sales',
  initialState: { history: [], loading: false, error: null, totalRevenue: 0 },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(completeSale.pending, (state) => { state.loading = true; state.error = null; })
      .addCase(completeSale.fulfilled, (state, action) => {
        state.loading = false;
        state.history.unshift(action.payload);
        state.totalRevenue += action.payload.total;
      })
      .addCase(completeSale.rejected, (state, action) => {
        state.loading = false;
        state.error = action.payload || 'Sale failed';
      });
  },
});

export default salesSlice.reducer;
```

📄 Step 3.2: Register & Build Sales Log

Update src/store/index.js:

```
import salesReducer from './slices/salesSlice';

export const store = configureStore({
  reducer: {
    inventory: inventoryReducer,
    cart: cartReducer,
    sales: salesReducer, // ✅ Add here
  },
});
```

```

    },
  });
};

```

Create `src/components/SalesLog.jsx`:

```

import { useDispatch, useSelector } from 'react-redux';
import { completeSale } from '../store/slices/salesSlice';
import { clearCart } from '../store/slices/cartSlice';

export default function SalesLog() {
  const dispatch = useDispatch();
  const { history, loading, error, totalRevenue } = useSelector(state => state.sales);

  const handleSale = async () => {
    const result = await dispatch(completeSale());
    if (completeSale.fulfilled.match(result)) {
      dispatch(clearCart()); // Clear UI cart on success
    }
  };

  return (
    <div className="mt-6 bg-white p-4 rounded shadow">
      <div className="flex justify-between items-center mb-3">
        <h2 className="font-bold">Sales History</h2>
        <div className="text-right">
          <p className="text-sm text-gray-500">Total Revenue</p>
          <p className="font-bold text-green-600">${totalRevenue.toFixed(2)}</p>
        </div>
      </div>
      <button
        onClick={handleSale}
        disabled={loading}
        className="w-full bg-green-600 text-white py-2 rounded mb-4 hover:bg-green-700
disabled:opacity-50"
        >
        {loading ? 'Processing...' : '✅ Complete Sale'}
      </button>
      {error && <p className="text-red-600 text-sm mb-2">⚠️ {error}</p>}
      <div className="max-h-40 overflow-y-auto space-y-2">
        {history.map(sale => (
          <div key={sale.id} className="border p-2 rounded text-sm">
            <p className="font-medium">Order #{sale.id} • {new
Date(sale.timestamp).toLocaleTimeString()}</p>
            <p className="text-gray-500">{sale.items.length} item(s) • ${sale.total.toFixed(2)}
          </p>
        </div>
        )
      )}
      </div>
    </div>
  );
}

```

✅ **Checkpoint:** Click "Complete Sale" → see loading → order appears in history → revenue updates → cart clears.
 Check DevTools → `sales/complete` lifecycle is clean.

◆ Phase 4: Pro Patterns & DevTools Mastery (25 mins)

🎯 Goal

Use memoized selectors for dashboard stats, handle edge cases, and debug like a senior dev.

📄 Memoized Dashboard Selector

```
// src/store/slices/salesSlice.js (add at bottom)
import { createSelector } from '@reduxjs/toolkit';

export const selectSalesMetrics = createSelector(
  [state => state.sales.history, state => state.sales.totalRevenue],
  (history, revenue) => ({
    totalSales: history.length,
    totalRevenue: revenue,
    avgOrderValue: history.length > 0 ? revenue / history.length : 0,
  })
);
```

Use in SalesLog.jsx :

```
import { selectSalesMetrics } from '../store/slices/salesSlice';
const metrics = useSelector(selectSalesMetrics);
// Use metrics.totalSales, metrics.avgOrderValue in UI
```

🔍 DevTools Workflow (Do This Live)

1. **Filter Actions:** Type `cart/` or `sales/` to isolate flows
2. **State Diff:** Click any action → see exact state changes in "Diff" tab
3. **Dispatch Manually:** Right-click state → "Dispatch" → run `restock` to test inventory recovery
4. **Time Travel:** Click previous states → watch UI revert instantly (no re-render bugs)

💡 **Production Rule:** Never put async logic in components. Keep it in thunks/RTK Query. Keep components focused on UI + dispatch.

Troubleshooting & Best Practices

Symptom	Likely Cause	Fix
State doesn't update	Missing slice in <code>configureStore</code>	Add reducer to <code>reducer: { ... }</code>
Component not re-rendering	Selector returns new object every call	Use <code>createSelector</code> or return primitives
Thunk never finishes	Missing <code>return</code> or unhandled promise	Ensure async function returns data or <code>rejectWithValue</code>
"State mutations detected" warning	Using <code>Date</code> , <code>Map</code> , or functions in state	Stick to JSON-safe types
Cart doesn't clear after sale	<code>clearCart</code> dispatched before thunk resolves	Wait for <code>fulfilled.match(result)</code> then clear

Professional Checklist

- ☐ Slices are small, focused, and export actions + reducer
- ☐ `useSelector` only subscribes to needed state (not entire store)
- ☐ Thunks handle `pending/fulfilled/rejected` cleanly
- ☐ DevTools Extension installed & verified
- ☐ No prop drilling between `ProductGrid`, `CartPanel`, `SalesLog`

Submission & Evaluation

✓ Deliverables

1. **GitHub Repository** with:

- Complete `store/slices/`, `components/`, `App.jsx`, `main.jsx`
- `README.md` with:
 - Setup instructions
 - Screenshot of POS + Sales Log
 - 1-paragraph reflection: *"How did Redux change your debugging workflow compared to `useState/Context`?"*

2. **Clean commit history** (min. 4: setup, slices, UI, async/sales)

Evaluation Criteria

Criteria	Points
✓ RTK store & slices configured correctly	25
✓ <code>useSelector</code> & <code>useDispatch</code> used properly	25
✓ Async thunk handles lifecycle & clears cart	20
✓ Memoized selectors & DevTools verified	15
✓ Clean, professional code structure	10
✓ Documentation & reflection	5
Total	100

Bonus (+10)

- 🛎 Add low-stock alert banner when `stock <= 5`
- 📦 Persist sales history to `localStorage` using Redux middleware or `subscribe`
- 🔄 Add "Undo Last Sale" using `history.pop()` + `restock()` dispatch
- 📊 Add date filter to sales log using `createSelector`

Instructor Note:

Redux isn't about writing more code. It's about writing **traceable** code. When your state flow is predictable, debuggable, and consistent, you ship faster, break less, and onboard teams effortlessly.

This POS dashboard mirrors real retail/e-commerce systems. Tomorrow, we'll refactor the async layer to **RTK Query** and show how modern teams cache 100+ endpoints without writing a single thunk.

Push your repo, tag `#MERNBootcamp-ReduxLab`, and I'll review your slice architecture, selector usage, and DevTools workflow. Keep state flat, actions explicit, and thunks clean.

— Kashif Raza | Predictable state, production-ready apps 🛒 📊